

Gravitational Search Algorithm: implementazione seriale e parallela su GPU (CUDA)

Fabio Zuliani

8 ottobre 2026

Sommario

In questo lavoro si realizza una versione parallela in CUDA C del Gravitational Search Algorithm, una metaeuristica di ottimizzazione ispirata alla legge di gravitazione, affiancata da una versione seriale in C che ne costituisce il riferimento algoritmico e l'oracolo di correttezza. Il problema affrontato è la massimizzazione di una funzione obiettivo polinomiale a coefficienti interi su variabili intere, in presenza di un dominio (hard constraint) e di un insieme di vincoli deboli (weak constraint) gestiti con il metodo della penalità. La parallelizzazione affida al device i due nuclei di calcolo più pesanti, cioè la valutazione della fitness con i vincoli e il calcolo delle forze. Poiché il calcolo delle forze domina il costo, esso è oggetto dello studio principale: se ne presentano **cinque versioni CUDA in progressione**, ciascuna aggiunge una tecnica di ottimizzazione (tiling in shared memory, decomposizione a livello di warp, riduzione con *warp shuffle*, accessi coalescenti e privi di conflitti di banco), e le si confronta sperimentalmente al variare della dimensione dell'istanza. La versione finale riduce il tempo del kernel delle forze da circa 3 a oltre 20 volte rispetto alla versione base, a seconda del numero di agenti. Della versione finale si studia inoltre la scalabilità forte e debole, interpretandone i risultati con le leggi di Amdahl e di Gustafson. Il confronto tra le cinque versioni, svolto su una Tesla T4, è stato ripetuto su una GeForce RTX 4060 dei laboratori dell'Università: il confronto mostra che il vantaggio relativo delle singole ottimizzazioni dipende dalla gerarchia di memoria e dalla capacità di calcolo in doppia precisione della GPU. La valutazione in aritmetica intera rende inoltre le due implementazioni numericamente identiche, il che consente una verifica di correttezza diretta.

Indice

1	Descrizione del problema affrontato	4
1.1	Introduzione	4
1.2	Il Gravitational Search Algorithm	5
1.3	Formulazione del problema	7
1.4	Algoritmo seriale	9
2	Metodologia adottata per la soluzione	9
2.1	Rappresentazione dei dati	10
2.2	Struttura della soluzione seriale (C)	10
2.3	Parallelizzazione su GPU (CUDA)	11
2.4	Le cinque versioni del kernel delle forze	12
3	Risultati sperimentali	15
3.1	Ambiente di test	15
3.2	Istanze di test	15
3.3	Confronto tra le cinque versioni del kernel delle forze	16
3.4	Scalabilità forte e debole	18
3.5	Confronto tra due GPU: Tesla T4 e GeForce RTX 4060	21
4	Valutazioni e osservazioni	24
4.1	Mini-guida all'uso	25

1 Descrizione del problema affrontato

1.1 Introduzione

L'obiettivo di questo studio è la realizzazione della versione parallela dell'algoritmo **GSA**. Negli ultimi decenni sono state sviluppate diverse metodologie euristiche per la risoluzione di problemi di ottimizzazione complessi, il motivo di tanto interesse in questo campo è dovuto al fatto che gli algoritmi di ottimizzazione classici non risolvono in maniera ottimale problemi di ottimizzazione con una dimensionalità piuttosto elevata dello spazio di ricerca. Questo problema è dovuto al fatto che lo spazio di ricerca aumenta esponenzialmente rispetto alla dimensione dell'input. Ciò provoca un decadimento delle prestazioni dei metodi di risoluzione classici, come per esempio la ricerca esaustiva.

Nella letteratura scientifica si può notare come negli ultimi anni sono comparsi diversi studi che presentano algoritmi i quali si ispirano a fenomeni naturali, i ricercatori hanno infatti dimostrato che essi possiedono ottime prestazioni per la risoluzione di problemi computazionalmente complessi come per l'appunto l'ottimizzazione di funzioni obiettivo. Alcuni esempi sono il Simulated Annealing, gli algoritmi genetici, l'Ant Colony Search Algorithm. Non esiste però un algoritmo che risolva ottimamente ogni problema di ottimizzazione, alcuni algoritmi forniscono soluzioni migliori per specifici problemi. Infatti la ricerca di nuovi algoritmi euristici per l'ottimizzazione è un problema ancora aperto.

Questa relazione riguarda l'implementazione e il confronto delle prestazioni della versione parallela e seriale di un algoritmo per l'ottimizzazione chiamato **Gravitational Search Algorithm**. Questo algoritmo si basa sulla legge di Gravità Newtoniana, l'idea alla base è: "Ogni particella nell'Universo attrae ogni altra particella con una forza che è direttamente proporzionale al prodotto delle masse e inversamente proporzionale al quadrato della distanza fra esse".

La **gravitazione** è la tendenza delle masse di accelerare una verso l'altra. Rappresenta una delle quattro interazioni fondamentali della Natura, la gravità è presente ovunque, l'inevitabilità della gravità la rende differente da ogni altra forza naturale. La **Forza Gravitazionale** tra due particelle è definita dalla seguente formula

$$F = G \frac{M_1 \times M_2}{R^2} \quad (1)$$

Dove **F** è il modulo della forza gravitazionale, **G** è la costante gravitazionale, **M₁**, **M₂** sono le masse delle rispettive particelle e **R** è la distanza tra esse. La seconda legge di Newton determina inoltre che, quando una forza **F** è applicata ad una particella, la sua accelerazione **a** è definita da

$$a = \frac{F}{m} \quad (2)$$

Ne consegue che tra tutte le particelle dell'Universo agisce una forza di attrazione gravitazionale, per la quale l'effetto è tanto maggiore quanto più grande e più vicina è la particella. Inoltre, a causa dell'effetto di diminuzione della gravità, il valore effettivo della costante di gravitazione universale dipende dall'effettiva età dell'Universo. Il comportamento di $\mathbf{G}$ è regolato da

$$G(t) = G(t_0) \times \left(\frac{t_0}{t}\right)^\beta, \quad \beta < 1 \quad (3)$$

La fisica teorica introduce tre tipologie di massa

- **Massa Gravitazionale Attiva $\mathbf{M}_a$.** Rappresenta la forza del campo gravitazionale dovuta ad un particolare oggetto.
- **Massa Gravitazionale Passiva $\mathbf{M}_p$.** Rappresenta la misura della forza dell'interazione di un oggetto con il campo gravitazionale.
- **Massa Inerziale $\mathbf{M}_i$.** Rappresenta la misura della resistenza di un oggetto al cambiamento dello stato di moto quando una forza è applicata.

Le Leggi di Newton possono ora essere scritte in funzione degli elementi introdotti precedentemente. La forza gravitazionale $\mathbf{F}_{ij}$, che agisce sulla massa $\mathbf{i}$ tramite $\mathbf{j}$, è proporzionale al prodotto di $\mathbf{M}_{aj}$ e $\mathbf{M}_{pi}$ e inversamente proporzionale al quadrato della distanza fra di essi.

$$F_{ij} = G \frac{M_{aj} \times M_{pi}}{R^2} \quad (4)$$

$$a_i = \frac{F_{ij}}{M_{ii}} \quad (5)$$

Terminata questa sezione introduttiva, che presenta l'euristica, la motivazione alla base dell'utilizzo di un fenomeno naturale e un breve richiamo alla fisica teorica, nella prossima sezione viene presentato l'algoritmo vero e proprio.

1.2 Il Gravitational Search Algorithm

Questa sezione è dedicata alla presentazione del **GSA**. Nell'algoritmo proposto, gli agenti sono considerati come oggetti e la loro prestazione è misurata dalla loro massa. Ogni oggetto attrae ogni altro agente mediante la forza di gravità, e questa forza causa un movimento globale degli agenti. Infatti le masse più pesanti corrispondono alle soluzioni migliori, le quali si muovono più lentamente. Questo garantisce il passo "sfruttamento" dell'algoritmo.

In **GSA** ogni agente è caratterizzato da quattro elementi:

- Posizione
- Massa Inerziale

- Massa Gravitazionale Attiva
- Massa Gravitazionale Passiva

La posizione dell'agente corrisponde ad una possibile soluzione del problema, e la sua massa gravitazionale e inerziale sono determinate attraverso la funzione fitness.

Con il passare del tempo ci si aspetta che le masse siano attratte dagli agenti più pesanti. L'agente più pesante rappresenterà dunque la soluzione ottima nello spazio di ricerca.

L'algoritmo GSA può essere considerato come un sistema isolato di masse, in cui ogni agente è soggetto a due tipi di forze Newtoniane

- Legge di gravità
- Legge del moto

Allo specifico istante $\mathbf{t}$, la forza che agisce sulla massa $\mathbf{i}$ tramite la massa $\mathbf{j}$ è definita come

$$F_{ij}^d = G(t) \frac{M_{pi}(t) \times M_{aj}(t)}{R_{ij} + \varepsilon} (x_j^d(t) - x_i^d(t)) \quad (6)$$

dove $R_{ij}(t) = \|\mathbf{x}_i(t), \mathbf{x}_j(t)\|_2$ è la distanza euclidea tra i due agenti ed ε è una piccola costante che previene la singolarità quando $R_{ij} \rightarrow 0$ (agenti coincidenti). Si osservi che, diversamente dalla legge di gravitazione classica, gli autori del GSA utilizzano R_{ij} e non R_{ij}^2 al denominatore, in quanto sperimentalmente fornisce risultati migliori.

Per introdurre una componente stocastica, la forza totale che agisce sull'agente i lungo la dimensione d è la somma pesata in modo casuale delle forze esercitate dagli altri agenti:

$$F_i^d(t) = \sum_{\substack{j \in K_{best} \\ j \neq i}} rand_j F_{ij}^d(t) \quad (7)$$

dove $rand_j$ è un numero casuale in $[0, 1]$ e K_{best} è l'insieme degli agenti con massa maggiore (le soluzioni migliori), la cui cardinalità decresce nel tempo. Per bilanciare esplorazione e sfruttamento, all'inizio tutti gli agenti esercitano forza (massima esplorazione), mentre col procedere delle iterazioni il loro numero diminuisce linearmente, così che alla fine solo gli agenti migliori influenzino gli altri (sfruttamento).

Per la seconda legge di Newton, l'accelerazione dell'agente i lungo la dimensione d è data dalla forza totale divisa per la sua massa inerziale:

$$a_i^d(t) = \frac{F_i^d(t)}{M_{ii}(t)} \quad (8)$$

Le masse sono calcolate a partire dalla fitness. Gli autori assumono, per semplicità, che le tre masse coincidano: $M_{ai} = M_{pi} = M_{ii} = M_i$. La massa di ciascun agente è ottenuta normalizzando la sua fitness rispetto alla migliore e alla peggiore della popolazione:

$$m_i(t) = \frac{fit_i(t) - worst(t)}{best(t) - worst(t)} \quad (9)$$

$$M_i(t) = \frac{m_i(t)}{\sum_{j=1}^N m_j(t)} \quad (10)$$

dove, nella formulazione originale a minimizzazione,

$$best(t) = \min_{j \in \{1, \dots, N\}} fit_j(t), \quad worst(t) = \max_{j \in \{1, \dots, N\}} fit_j(t) \quad (11)$$

In questo modo l'agente migliore ha massa normalizzata pari a 1 e il peggiore pari a 0: le soluzioni migliori risultano più "pesanti" e attraggono le altre. La costante gravitazionale G non è fissa, ma decresce nel tempo per ridurre progressivamente l'intensità dell'attrazione e favorire la convergenza. Nell'implementazione si adotta la seguente forma esponenziale decrescente:

$$G(t) = G_0 e^{-\alpha t/T} \quad (12)$$

con G_0 valore iniziale, α costante di decadimento, t iterazione corrente e T numero totale di iterazioni.

Infine, la legge del moto aggiorna prima la velocità e poi la posizione di ogni agente. La nuova velocità è una frazione casuale della velocità precedente sommata all'accelerazione; la nuova posizione si ottiene sommando la velocità aggiornata:

$$v_i^d(t+1) = rand_i v_i^d(t) + a_i^d(t) \quad (13)$$

$$x_i^d(t+1) = x_i^d(t) + v_i^d(t+1) \quad (14)$$

Il termine casuale $rand_i \in [0, 1]$ sulla velocità conferisce all'esplorazione la natura stocastica tipica delle metaeuristiche. Il processo si ripete finché non si raggiunge il criterio di arresto (nella nostra implementazione, un numero prefissato di iterazioni T).

1.3 Formulazione del problema

Il GSA viene applicato a un problema di **ottimizzazione vincolata a variabili intere**. Una possibile soluzione è un vettore K -dimensionale $\mathbf{x} = (x_1, \dots, x_K)$, che nell'algoritmo corrisponde alla posizione di un agente. Su indicazione del docente il fulcro del progetto è la parallelizzazione : si assume pertanto che la funzione obiettivo f e i vincoli g_j siano semplici **polinomi a coefficienti interi** e che le variabili assumano **valori interi**.

Questa scelta rende la valutazione interamente in aritmetica intera, con due conseguenze utili: le due implementazioni producono fitness identiche (aspetto sfruttato per la verifica di correttezza) e il costo di valutazione dipende soltanto dalla struttura dei polinomi e dalla taglia dell'istanza. Il problema si compone di tre elementi: un dominio (hard constraint), una funzione obiettivo e un insieme di vincoli deboli (weak constraint) gestiti a penalità.

Dominio (hard constraint). Ogni variabile è vincolata a un intervallo intero $x_d \in \{lb_d, \dots, ub_d\}$, per $d = 1, \dots, K$. Nell'implementazione si adotta un dominio uniforme $[LO, HI]$ con $LO = -5$ e $HI = 5$. Si tratta di un vincolo *forte* che non può mai essere violato: viene garantito per costruzione riportando entro gli estremi ogni coordinata che ne esca (clamping) a valle di ogni aggiornamento di posizione, e arrotondando la posizione all'intero più vicino in fase di valutazione.

Rappresentazione dei polinomi. Sia la funzione obiettivo sia ciascun vincolo sono polinomi a coefficienti interi, espressi come somma di monomi. Ogni polinomio è formato da NT termini e ogni termine è il prodotto di un coefficiente intero per DEG variabili, con indici eventualmente ripetuti in modo da rappresentare le potenze:

$$p(\mathbf{x}) = \sum_{t=1}^{NT} c_t \prod_{\ell=1}^{DEG} x_{v_{t,\ell}}, \quad c_t \in \mathbb{Z}, \quad v_{t,\ell} \in \{1, \dots, K\}. \quad (15)$$

I coefficienti c_t e gli indici $v_{t,\ell}$ sono generati da un seme dedicato: a parità di seme le funzioni restano fisse mentre si fa variare la taglia dell'istanza, condizione necessaria per un confronto equo dei tempi.

Funzione obiettivo. La funzione obiettivo $f(\mathbf{x})$ è un polinomio della forma precedente, da *massimizzare* sul dominio intero.

Vincoli deboli (weak constraint). Al problema si aggiunge un insieme di NC vincoli $g_j(\mathbf{x}) > 0$, ciascuno un polinomio a coefficienti interi, che *possono* essere violati a un costo. Si adotta il **metodo della penalità statica**: a ogni vincolo violato è associata una penalità intera costante P_j , fissata a priori e non adattata durante le iterazioni. Si noti che nel testo del progetto il numero di vincoli è indicato con N ; poiché nel GSA N denota il numero di agenti, nel seguito il numero di vincoli è indicato con NC per evitare ogni ambiguità.

Fitness scalare. I tre elementi vengono ricondotti a un'unica funzione scalare da massimizzare, sottraendo alla funzione obiettivo la somma delle penalità dei vincoli violati:

$$\Phi(\mathbf{x}) = f(\mathbf{x}) - \sum_{j=1}^{NC} P_j \chi_j(\mathbf{x}), \quad \chi_j(\mathbf{x}) = \begin{cases} 1 & \text{se } g_j(\mathbf{x}) \leq 0 \text{ (violato)} \\ 0 & \text{altrimenti} \end{cases} \quad (16)$$

Massimizzare Φ significa cercare un punto del dominio intero che renda grande la funzione obiettivo e insieme violi il minor peso possibile di vincoli.

È questa la funzione di fitness che il GSA usa per assegnare le masse agli agenti.

1.4 Algoritmo seriale

Algorithm 1: Gravitational Search Algorithm

Input: N agenti, K dimensioni, T iterazioni, dominio $\mathcal{D}$, parametri G_0, α

Output: migliore soluzione trovata $\mathbf{x}^*$

Inizializza le posizioni $\mathbf{x}_i$ a caso in $\mathcal{D}$ e le velocità $\mathbf{v}_i \leftarrow \mathbf{0}$;

for $t \leftarrow 1$ **to** T **do**

 Valuta la fitness $\Phi(\mathbf{x}_i)$ di ogni agente;

 Aggiorna $best(t)$ e $worst(t)$;

$G(t) \leftarrow G_0 e^{-\alpha t/T}$;

 Calcola le masse $M_i(t)$ (normalizzate su $best$ e $worst$);

$K_{best} \leftarrow$ insieme dei migliori agenti (decrescente in t);

foreach agente i **do**

$\mathbf{F}_i \leftarrow$ forza totale dagli agenti in K_{best} ; // eq. forza

$\mathbf{a}_i \leftarrow \mathbf{F}_i / M_i(t)$;

$\mathbf{v}_i \leftarrow rand \cdot \mathbf{v}_i + \mathbf{a}_i$;

$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$;

 Riporta $\mathbf{x}_i$ nel dominio $\mathcal{D}$; // hard constraint

return l'agente con fitness migliore;

2 Metodologia adottata per la soluzione

Entrambe le versioni, seriale e parallela, implementano lo stesso identico algoritmo descritto nella Sezione 1.2: la versione seriale è scritta in C, mentre quella parallela in CUDA C. La versione seriale svolge il ruolo di riferimento algoritmico e di oracolo di correttezza: poiché la valutazione avviene in aritmetica intera, produce le stesse identiche fitness della versione CUDA e ne consente la verifica. L'obiettivo non è modificare l'euristica, ma studiare la parallelizzazione sulla GPU e, in particolare, ottimizzare il nucleo di calcolo dominante confrontando fra loro diverse versioni CUDA al variare della dimensione del problema. Il GSA si presta bene alla parallelizzazione: a ogni iterazione gli agenti vengono aggiornati in modo largamente indipendente l'uno dall'altro, ed è proprio questa indipendenza che viene sfruttata sulla GPU.

Una precisazione sulla convenzione di ottimizzazione. La descrizione canonica del GSA della Sezione 1.2, tratta dall'articolo originale, riguarda un problema di minimizzazione. Il problema qui affrontato è invece di massimizzazione della fitness Φ . L'adattamento è immediato e non modifica l'algoritmo: si

identifica la massa maggiore con la fitness maggiore, ponendo $best(t)$ pari al valore massimo della fitness (l'agente migliore) e $worst(t)$ al valore minimo. La normalizzazione delle masse resta coerente, poiché all'agente migliore corrisponde comunque massa normalizzata pari a 1.

2.1 Rappresentazione dei dati

In entrambe le versioni la popolazione è memorizzata in **array monodimensionali contigui**, allocati in ordine *row-major*. La matrice logica $N \times K$ delle posizioni (dove N è il numero di agenti e K la dimensione del problema) è quindi appiattita in un unico vettore, in cui la coordinata d dell'agente i si trova in posizione $i \cdot K + d$.

Le principali strutture dati sono:

- il vettore delle posizioni $\mathbf{x}$ di dimensione $N \cdot K$;
- il vettore delle velocità $\mathbf{v}$ di dimensione $N \cdot K$;
- i vettori scalari per agente: fitness e massa, entrambi di dimensione N ;
- le tabelle dei coefficienti e degli indici dei polinomi (di f e degli NC vincoli), di tipo intero, generate una sola volta all'avvio.

La scelta di un layout piatto, anziché di una matrice di puntatori, è motivata dall'efficienza: la memoria è contigua, l'allocazione sul device richiede una singola `cudaMalloc` per array e i trasferimenti fra host e device avvengono in un unico blocco. Mantenere lo stesso layout nelle due versioni rende inoltre il porting diretto e i risultati direttamente confrontabili.

2.2 Struttura della soluzione seriale (C)

La versione seriale ricalca fedelmente lo pseudocodice della Sezione 1.4. Dopo l'inizializzazione casuale delle posizioni nel dominio e l'azzeramento delle velocità, il programma esegue il ciclo principale per T iterazioni. A ogni iterazione si valuta la fitness Φ di ogni agente, si individuano il migliore e il peggiore, si calcolano le masse normalizzate, si seleziona l'insieme K_{best} e infine, per ogni agente, si calcolano forza e accelerazione e si aggiornano velocità e posizione, applicando il clamping al dominio.

Il costo computazionale ha due contributi principali. Il **calcolo delle forze** è dominante al crescere di N : ogni agente interagisce con tutti gli agenti in K_{best} lungo tutte le K dimensioni, per una complessità di $O(N^2 \cdot K)$ per iterazione. La **valutazione della fitness** costa invece $O(N \cdot NC \cdot NT \cdot DEG)$ per iterazione e diventa a sua volta onerosa quando il numero di vincoli NC è grande. Tutti gli altri passi (riduzioni, ordinamento per la selezione di K_{best} , aggiornamento del moto) hanno costo lineare o quasi lineare in N . Sono dunque il calcolo delle forze e la valutazione dei vincoli i due *colli di bottiglia*, ed entrambi sono candidati naturali alla parallelizzazione.

2.3 Parallelizzazione su GPU (CUDA)

Tipo di parallelismo. Si adotta un parallelismo *data-parallel* secondo il modello SIMT, con la strategia **un thread per agente**. Poiché a ogni iterazione ciascun agente calcola la propria forza, la propria accelerazione e il proprio spostamento in modo indipendente dagli altri, la corrispondenza tra thread e agente è la più naturale: il thread di indice globale $i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$ si occupa dell'agente i . Ogni thread è l'unico scrittore dei dati del proprio agente, quindi non sono necessarie né operazioni atomiche né sincronizzazioni tra thread. Questa è la strategia della versione base; le versioni F3–F5 del kernel delle forze assegnano invece un intero warp a ciascun agente (Sezione 2.4).

Suddivisione tra host e device. Non tutto l'algoritmo viene portato sulla GPU. Sul *device* si esegue la parte computazionalmente pesante: la valutazione della fitness, il calcolo di forze e accelerazioni e l'aggiornamento di velocità e posizioni. Sull'*host* restano invece la generazione dei numeri casuali, le riduzioni per *best*, *worst* e la somma delle masse, e la selezione di K_{best} tramite ordinamento. Questa suddivisione è motivata da due ragioni: la prima è di semplicità implementativa, poiché si evitano librerie esterne come cuRAND e primitive di ordinamento su GPU; la seconda è di natura prestazionale, poiché queste operazioni hanno costo $O(N)$ o $O(N \log N)$, trascurabile rispetto ai due nuclei pesanti, e i relativi trasferimenti sono anch'essi $O(N)$.

Kernel implementati. Il calcolo sul device è affidato a tre kernel distinti, lanciati in sequenza a ogni iterazione:

- **Valutazione della fitness.** Ogni thread calcola la fitness Φ del proprio agente, valutando la funzione obiettivo e gli NC vincoli sulla posizione arrotondata all'intero, in aritmetica intera. La complessità è $O(NC \cdot NT \cdot DEG)$ per thread: quando NC è grande questo kernel diventa un secondo fronte di calcolo pesante accanto a quello delle forze. Le tabelle dei coefficienti dei polinomi risiedono in global memory e sono lette da tutti i thread. Il vettore delle fitness viene poi trasferito all'*host*, dove avvengono le riduzioni per *best*, *worst* e per la somma delle masse.
- **Calcolo delle forze e accelerazione.** Ogni thread, per il proprio agente i , accumula in una variabile locale la forza totale esercitata dagli agenti in K_{best} lungo ciascuna dimensione, e ne ricava l'accelerazione dividendo per la massa. È il kernel dominante al crescere di N , con complessità $O(N \cdot K)$ per thread. Essendo il collo di bottiglia principale, di questo kernel si realizzano cinque versioni progressivamente ottimizzate, presentate nella Sezione 2.4.
- **Aggiornamento del moto.** Ogni thread aggiorna velocità e posizione del proprio agente secondo la legge del moto e applica il clamping al

dominio (hard constraint). È un kernel distinto da quello delle forze, lanciato subito dopo di esso.

Gestione della memoria. Gli array sul device sono allocati dinamicamente con `cudaMalloc` e i trasferimenti avvengono con `cudaMemcpy` in modalità *pageable*. Le tabelle dei polinomi vengono trasferite una sola volta a inizio esecuzione, poiché non cambiano durante le iterazioni. A ogni iterazione l'host calcola le grandezze che gli competono (masse, K_{best} , coefficienti casuali) e le trasferisce sul device, che a sua volta restituisce all'host il vettore delle fitness. I lanci di kernel sono asincroni; la sincronizzazione è garantita dalle `cudaMemcpy` bloccanti e da una `cudaDeviceSynchronize` finale prima della misurazione dei tempi.

Accesso alla memoria. Nel kernel delle forze i due accessi principali hanno comportamento diverso rispetto alla coalescenza. La lettura della posizione dell'agente sorgente j è *uguale per tutti i thread di un warp* (stessa sorgente nello stesso istante) e viene quindi servita come *broadcast*, in modo efficiente. La lettura della posizione dell'agente *proprio* di ciascun thread, invece, con il layout row-major fa sì che thread consecutivi di un warp accedano a indirizzi distanti K elementi: l'accesso non è pienamente coalescente. Un diverso mapping tra thread e dati, che rende coalescenti anche questi accessi, è realizzato nella versione F5 (Sezione 2.4).

Dettagli implementativi. La gestione degli errori CUDA è centralizzata nella macro `HANDLE_ERROR` e nella funzione `checkCUDAError`, mentre le proprietà del device si ottengono con `cudaGetDeviceProperties`. I parametri N , K , NC , DEG , T e la dimensione del blocco sono resi sovrascrivibili da riga di comando tramite direttive `-D` in fase di compilazione, così da poter eseguire i benchmark al variare della configurazione senza modificare il sorgente.

2.4 Le cinque versioni del kernel delle forze

Il calcolo delle forze è il nucleo dominante dell'algoritmo, con costo $O(N^2 \cdot K)$ per iterazione: è quindi la porzione più utile da parallelizzare, e su di essa si concentra lo studio principale. Si realizzano cinque versioni CUDA dello stesso kernel, in **progressione**: ciascuna calcola le stesse accelerazioni (a meno, per F3 e F4, dell'ordine delle somme in virgola mobile), ma aggiunge una tecnica di ottimizzazione, così da isolarne il contributo. Tutte partono da un interrogativo: dato che ogni agente bersaglio interagisce con gli *stessi* agenti sorgente in K_{best} , come si organizza al meglio il lavoro tra i thread e la gerarchia di memoria?

F1, baseline (global memory). È la versione di partenza: un thread per agente, secondo la strategia descritta nella Sezione 2.3. Ogni thread, per il proprio agente, scorre tutti i sorgenti in K_{best} , ne rilegge la posizione dalla global memory, calcola la distanza R_{ij} e accumula i contributi di forza in un array locale `acc_1[K]`. Il limite è duplice: le posizioni dei sorgenti vengono

rilette dalla memoria globale una volta per ogni thread (letture ridondanti), e con soli N thread la GPU, che dispone di migliaia di unità di calcolo, resta ampiamente sottoutilizzata finché N è piccolo. Con blocchi da 256 thread, per tutte le taglie studiate ($N \leq 6400$) la baseline lancia al più 25 blocchi, meno dei 40 multiprocessori (SM) della Tesla T4.

F2, tiling in shared memory. La prima leva agisce sul traffico di memoria. Poiché tutti gli agenti bersaglio leggono gli stessi sorgenti, le posizioni dei sorgenti si caricano a blocchi (*tile*) di dimensione **TILE** nella **shared memory**, una memoria interna al chip condivisa dai thread di uno stesso blocco e molto più veloce della global, e vengono riusate da tutti i thread del blocco. Ogni sorgente viene così letto dalla global una volta per blocco anziché una volta per warp (all'interno di un warp la lettura della baseline è già un broadcast), con una riduzione del traffico pari al numero di warp per blocco, cioè 8 con blocchi da 256 thread. Nel ciclo sui sorgenti del tile tutti i thread leggono lo stesso elemento della shared memory, che viene quindi servito in broadcast senza conflitti. Il prezzo è la pressione sui registri: ogni thread mantiene sia la propria posizione sia l'accumulatore ($2K = 128$ valori **double**), che eccedono i registri disponibili e vengono in parte spostati in local memory (608 byte per thread, rilevati con l'opzione `-Xptxas -v` di `nvcc`). La decomposizione resta a un thread per agente, quindi F2 misura il solo effetto del riuso in shared. Il caricamento del tile è cooperativo e richiede una barriera `__syncthreads()` dopo il caricamento (il tile deve essere completo prima dell'uso) e una dopo l'uso (prima di sovrascriverlo); i thread con indice oltre N devono comunque raggiungere le barriere, pena lo stallo del blocco. La dimensione del tile è un compromesso tra riuso e capacità: la shared è limitata (48 KB per blocco sulla Tesla T4) e l'array dei tile occupa $\text{TILE} \cdot K \cdot 8$ byte.

F3, decomposizione a warp e riduzione con shuffle. La seconda leva agisce sulla decomposizione. Nella baseline l'accumulo dei contributi dei sorgenti è una *riduzione seriale* eseguita da un solo thread; qui si assegna invece **un intero warp** (32 thread, dette *lane*) a ciascun agente, e le lane si spartiscono i sorgenti in K_{best} . Ogni lane calcola la distanza R_{ij} una sola volta per il proprio sorgente e accumula un contributo parziale; alla fine, i contributi parziali delle 32 lane vengono sommati con una **riduzione a livello di warp**, realizzata tramite la primitiva `__shfl_down_sync`. Questa istruzione permette a una lane di leggere direttamente un registro di un'altra lane dello stesso warp, senza passare dalla shared memory e senza barriere: la somma dei 32 valori richiede $\log_2 32 = 5$ passi. Il vantaggio principale è che il parallelismo passa da N a $32N$ thread: anche con pochi agenti il lavoro si distribuisce su tutti gli SM della GPU, mentre la baseline ne usa solo una parte. L'occupazione di ciascun SM resta invece limitata: l'accumulatore parziale di ogni lane è ancora un array di K valori **double**, tenuto nei registri, per cui su ogni SM risiede un solo blocco da 8 warp, cioè un quarto del massimo. Inoltre, poiché le lane si spartiscono i sorgenti, nello stesso istante leggono righe diverse della matrice delle posizioni, distanti $K \cdot 8 = 512$

byte: l'accesso alla global memory non è coalescente. Si osservi che, essendo l'accumulo delle forze in virgola mobile (`double`), l'ordine diverso delle somme può produrre differenze dell'ordine dell'ultima cifra rispetto alla baseline; la fitness finale, calcolata in aritmetica intera, resta invariata.

F4, warp con shuffle e tiling combinati. Questa versione unisce le due leve. Si mantiene la decomposizione a warp con riduzione shuffle di F3, ma i sorgenti vengono caricati a tile nella shared memory come in F2. Gli (`BLOCK/32`) warp di uno stesso blocco corrispondono ad agenti diversi che però attraggono verso gli stessi sorgenti: il tile, caricato una volta per blocco, è dunque riusato da tutti i warp del blocco, sommando il vantaggio della decomposizione a warp (F3) a quello del ridotto traffico di memoria (F2). Resta però un difetto nell'accesso alla shared: poiché le lane si spartiscono i sorgenti, nello stesso istante leggono righe diverse del tile, distanti $K = 64$ valori `double`. Tutti questi indirizzi cadono nello stesso banco della shared memory e le letture vengono serializzate (*bank conflict*).

F5, warp per agente con lane sulle dimensioni. L'ultima versione nasce da un'osservazione sulle precedenti: F2 accede alla memoria nel modo migliore (tutti i thread leggono lo stesso sorgente, in broadcast) ma usa pochi SM, mentre F3 e F4 usano tutti gli SM ma con accessi sfavorevoli. F5 mantiene un warp per agente e il tile in shared, ma cambia il ruolo delle lane. Per ogni tile si svolgono tre passi:

- *caricamento cooperativo del tile*, in cui thread consecutivi leggono coordinate consecutive dello stesso sorgente, con accesso coalescente alla global memory;
- *fase A (lane $\leftrightarrow$ sorgente)*: la lane t calcola la distanza R e il coefficiente $rand_j G M_j / (R + \varepsilon)$ del sorgente t , una sola volta per coppia (agente, sorgente). Il tile è memorizzato con passo $K + 1$ anziché K (*padding*), così che lane che leggono righe diverse cadano su banchi diversi;
- *fase B (lane $\leftrightarrow$ dimensione)*: il warp scorre i sorgenti nello stesso ordine della baseline e la lane ℓ aggiorna le dimensioni ℓ e $\ell + 32$; le letture in shared sono contigue e il coefficiente è letto in broadcast.

L'accumulatore si riduce a $K/32 = 2$ registri per lane, non serve alcuna riduzione finale con shuffle e, grazie al minor uso di registri, su ogni SM possono risiedere due blocchi anziché uno. Poiché l'ordine di accumulo e le espressioni sono quelli della baseline, F5 calcola le stesse accelerazioni di F1; ciò è stato verificato con un'emulazione su CPU dei passi del kernel, che riproduce bit a bit i risultati della baseline. È la versione più efficiente a tutte le taglie misurate ed è quella impiegata nella configurazione finale.

La Sezione 3.3 confronta sperimentalmente le cinque versioni al variare di N , misurando direttamente il tempo del solo kernel delle forze tramite gli eventi CUDA.

3 Risultati sperimentali

3.1 Ambiente di test

Le prove sono state eseguite nell'ambiente Google Colab. La GPU è una NVIDIA Tesla T4, architettura Turing, compute capability 7.5. La parte seriale gira sulla CPU della stessa sessione Colab (Intel Xeon a due vCPU) con circa 13 GB di RAM, sotto sistema operativo Ubuntu. Il codice seriale è compilato con gcc, opzioni `-O2 -std=gnu99`; il codice CUDA con nvcc, opzioni `-O2 -arch=sm_75`. Il modello esatto di CPU e la versione di CUDA disponibili nella sessione si rilevano con i comandi `nvidia-smi` e `nvcc --version` e vanno riportati qui a partire dall'ambiente effettivo usato. Il confronto tra le cinque versioni del kernel delle forze è stato poi ripetuto su una seconda GPU, una GeForce RTX 4060 dei PC dell'Università: l'ambiente e il confronto tra le due GPU sono descritti nella Sezione 3.5.

Componente	Configurazione
GPU	NVIDIA Tesla T4 (Turing, cc 7.5)
Risorse GPU	40 SM, 16 GB GDDR6, cache L2 da 4 MB
Doppia precisione	1/32 della singola (circa 0,25 TFLOPS)
CPU	Intel Xeon, sessione Google Colab
RAM	circa 13 GB
Sistema	Ubuntu (Google Colab)
Compilatori	gcc <code>-O2 -std=gnu99</code> , nvcc <code>-O2 -arch=sm_75</code>

Tabella 1: Ambiente hardware e software di test.

3.2 Istanze di test

Prima di riportare i risultati è utile precisare che cosa si intende per *istanza* del problema. Un'istanza è la combinazione di due elementi: la **taglia**, cioè i parametri (N, K, NC, DEG) che ne fissano le dimensioni, e le **funzioni** f e g_j , cioè i polinomi che definiscono l'obiettivo e i vincoli. Questi polinomi non vengono scelti a mano, ma **generati da un seme** (CSEED nel codice): il seme inizializza il generatore pseudocasuale che produce i coefficienti interi e gli indici delle variabili di ogni monomio. Il seme, insieme alla taglia, definisce quindi completamente l'istanza: a parità di seme e di taglia le funzioni sono sempre le stesse, e l'esecuzione è riproducibile.

Questa proprietà è ciò che rende equo il confronto delle prestazioni. Poiché il seme è fisso, si può **far variare la sola dimensione** del problema (in particolare il numero di agenti N) mantenendo *identiche* le funzioni f e g_j : le curve dei tempi al variare di N misurano così l'effetto della taglia, e non un cambiamento del problema sottostante. Si noti che al crescere di N cambia

il numero di agenti, ma non le funzioni: nuovi agenti sono semplicemente nuove posizioni candidate nello stesso problema. Il numero di termini per polinomio è posto pari a K e il grado dei monomi è DEG ; il dominio è $[-5, 5]$ su ogni variabile.

Per la valutazione delle prestazioni si utilizza una taglia volutamente onerosa, con $K = 64$, $NC = 512$ e $DEG = 3$, così che entrambi i kernel, quello della fitness e quello delle forze, rappresentino un carico significativo. Con $NC = 512$ le tabelle dei coefficienti occupano circa mezzo megabyte in global memory. Il numero di agenti N è la grandezza fatta variare per lo studio di scalabilità e per il confronto tra le cinque versioni del kernel delle forze, poiché, come detto, non altera le funzioni.

3.3 Confronto tra le cinque versioni del kernel delle forze

Questa sezione è il cuore dello studio sulla parallelizzazione: si confrontano le cinque versioni CUDA del kernel delle forze descritte nella Sezione 2.4. Il confronto è *a parità di tutto il resto*, cioè con la stessa dinamica del GSA e la stessa versione degli altri kernel: cambia soltanto l'implementazione del calcolo delle forze. Per isolare il costo del solo kernel, invece del tempo totale si misura direttamente il tempo del kernel delle forze tramite gli **eventi CUDA**, sommato sulle iterazioni e riportato come tempo medio per iterazione. Tutte le misure provengono dalla stessa sessione, poiché i tempi assoluti variano tra sessioni sulla GPU condivisa mentre il rapporto tra le versioni resta stabile. La Tabella 2 e la Figura 1 riportano i risultati al variare di N . I dati mostrano comportamenti distinti per ciascuna leva

N	F1	F2	F3	F4	F5	F1/F5
100	0,730	0,568	0,103	0,131	0,070	10,4×
200	2,569	1,996	0,217	0,165	0,115	22,3×
400	5,286	4,112	0,719	0,555	0,377	14,0×
800	10,871	7,851	1,882	1,377	1,182	9,2×
1600	21,075	15,739	6,010	4,460	3,340	6,3×
3200	40,763	30,364	22,496	16,001	12,710	3,2×
6400	145,518	66,880	93,018	63,314	51,115	2,8×

Tabella 2: Tempo del solo kernel delle forze (ms per iterazione, mediana di 2 esecuzioni da 100 iterazioni) per le cinque versioni CUDA, al variare di N , e speedup della versione finale F5 rispetto alla baseline F1. F1 baseline, F2 tiled, F3 warp+shuffle, F4 warp+shuffle+tiled, F5 warp con lane sulle dimensioni + tiled. Taglia $K = 64$, $NC = 512$, $DEG = 3$.

di ottimizzazione.

Il **tiling** (F2) riduce il tempo della baseline di un fattore pressoché costante, tra 1,2 e 1,4, fino a $N = 3200$: il traffico verso la global memory diminuisce,

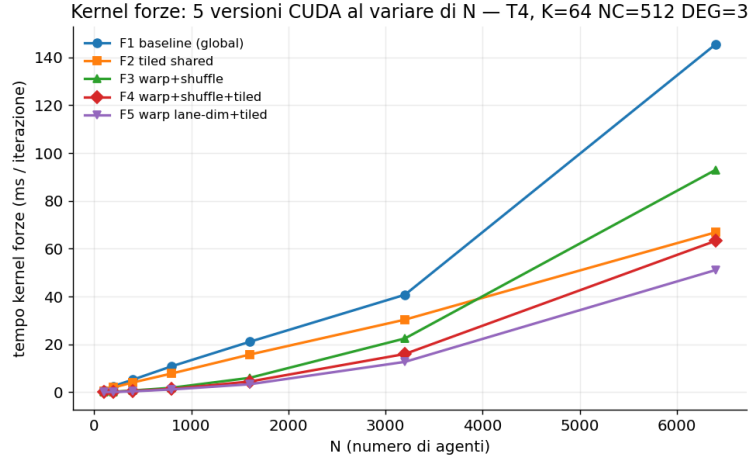


Figura 1: Tempo del kernel delle forze al variare di N per le cinque versioni CUDA.

ma la decomposizione è la stessa e la GPU resta sottoutilizzata allo stesso modo.

La **decomposizione a warp con riduzione shuffle** (F3) produce il salto maggiore alle taglie piccole (12,7 volte più veloce della baseline a $N = 200$), perché distribuisce il lavoro su tutti gli SM. Il vantaggio però si riduce al crescere di N , fino a 1,6 volte a $N = 6400$. Il motivo è il diverso andamento delle curve. La baseline usa al più 25 dei 40 SM: aggiungere agenti attiva altri SM, e il suo tempo cresce circa linearmente in N (raddoppia a ogni raddoppio di N fino a 3200). F3, che occupa già tutta la GPU, cresce invece con il lavoro totale, cioè come N^2 (circa 4 volte a ogni raddoppio). Il buon andamento della baseline alle taglie grandi è quindi solo apparente: dipende dal fatto che lascia inutilizzata una parte della GPU.

La **combinazione** di warp e tiling (F4) rende la versione circa il 25–30% più veloce di F3 da $N = 200$ in poi. A $N = 100$ le due si equivalgono: con pochi agenti la sincronizzazione aggiuntiva non è ripagata dal riuso del tile. La versione con **lane sulle dimensioni** (F5) è la più veloce a tutte le taglie: da 1,2 a 1,9 volte rispetto a F4 e da 2,8 a 22 volte rispetto alla baseline. Guadagna di più alle taglie piccole, perché elimina la riduzione finale con shuffle di F3 e F4 (64 riduzioni da 5 passi per agente), un costo fisso che pesa molto quando i sorgenti sono pochi.

Fa eccezione al quadro generale il passaggio da $N = 3200$ a $N = 6400$: il tempo della baseline cresce di 3,5 volte invece che di 2, e F2 supera F3. La spiegazione più coerente con i dati è la capienza della cache L2, che sulla Tesla T4 è di 4 MB. Il kernel delle forze legge l'array delle posizioni e scrive quello delle accelerazioni, ciascuno di $N \cdot K \cdot 8$ byte: a $N = 3200$ i due array occupano in tutto circa 3,3 MB e risiedono in L2, a $N = 6400$ ne occupano

6,6 MB e non vi stanno più, quindi una parte delle letture della baseline deve raggiungere la memoria globale. F2, che legge ogni sorgente una sola volta per blocco e poi lavora in shared memory, ne risente molto meno e cresce di un fattore 2. Il confronto con una GPU dotata di una L2 molto più grande, nella Sezione 3.5, conferma questa interpretazione.

Le cinque versioni producono la stessa soluzione. Le varianti F2 e F5, che mantengono l'ordine di accumulo della baseline, coincidono con essa bit a bit; le varianti F3 e F4, che sommano i contributi in ordine diverso, possono differire per l'ultima cifra a causa della natura non associativa della virgola mobile, ma la fitness finale, calcolata in aritmetica intera, resta identica in tutte le versioni. La correttezza rispetto al riferimento è verificata dal fatto che, sull'istanza di prova, tutte le versioni CUDA e la versione seriale raggiungono la stessa migliore fitness (il valore intero stampato a fine esecuzione coincide), come atteso dato che la valutazione di funzione obiettivo e vincoli è interamente in aritmetica intera.

3.4 Scalabilità forte e debole

Oltre al confronto tra le versioni, si studia la scalabilità della versione finale F5, cioè la capacità del programma di esprimere maggior potenza computazionale all'aumentare delle risorse impiegate. Si misurano le due forme di scalabilità, forte e debole, e se ne interpretano i risultati con le leggi di Amdahl e di Gustafson.

Speedup ed efficienza. Indicando con T_P il tempo di computazione ottenuto utilizzando P processori, lo speedup e l'efficienza di parallelizzazione, cioè il rapporto tra lo speedup effettivo e quello ideale, sono definiti come

$$S(P) = \frac{T_1}{T_P}, \quad E(P) = \frac{S(P)}{P} \quad (17)$$

La situazione ideale è lo speedup lineare, $S(P) = P$, a cui corrisponde un'efficienza unitaria. Si noti che T_1 è il tempo dello *stesso* programma parallelo eseguito su un solo processore, e non quello della versione seriale, che è un programma diverso eseguito su un'architettura diversa.

Legge di Amdahl e scalabilità forte. Si suddivide il tempo di computazione in una porzione seriale s e in una porzione parallelizzabile p , con $s + p = 1$. Secondo la legge di Amdahl, utilizzando P processori lo speedup ottenibile è

$$S(P) = \frac{1}{s + \frac{p}{P}} \xrightarrow{P \rightarrow \infty} \frac{1}{s} \quad (18)$$

quindi, per un problema di dimensione fissata, lo speedup è limitato dalla sua porzione seriale. Lo *strong scaling* esprime come varia il tempo di computazione al variare del numero di processori, per una dimensione totale fissa del problema; idealmente $T_P = T_1/P$. Invertendo la legge, da ogni

speedup misurato si ricava la porzione seriale che lo spiega:

$$s = \frac{P/S(P) - 1}{P - 1} \quad (19)$$

Se il programma segue la legge di Amdahl, il valore di s così stimato resta costante al variare di P .

Legge di Gustafson e scalabilità debole. La legge di Gustafson assume che la porzione parallelizzabile cresca linearmente con il numero di processori, cioè che tutti i processori svolgano la stessa quantità di lavoro, e che la porzione seriale non vari con la dimensione del problema. Lo speedup che ne risulta, detto *scaled speedup*, è

$$S(P) = s + p \cdot P = P - s(P - 1) \quad (20)$$

In questo caso il parallelismo non serve a diminuire il tempo impiegato per svolgere un dato compito, ma ad aumentare la quantità di lavoro svolta nell'unità di tempo. Il *weak scaling* misura il tempo di computazione mentre la dimensione del problema cresce linearmente con il numero di processori: idealmente il tempo resta costante e lo speedup scalato è pari a P .

Processori e misura. Nella GPU l'unità di calcolo indipendente è lo Streaming Multiprocessor (SM); la Tesla T4 ne ha 40, raggruppati a coppie in 20 *Texture Processing Cluster* (TPC). Per far variare il numero di processori a disposizione del programma si usa il Multi-Process Service (MPS) di NVIDIA, che tramite la variabile d'ambiente `CUDA_MPS_ACTIVE_THREAD_PERCENTAGE` limita la frazione della GPU utilizzabile da un processo. MPS assegna gli SM a coppie, quindi l'unità minima è il TPC: si adotta pertanto il TPC come processore, con $P \in \{1, 2, 5, 10, 20\}$, pari a 2, 4, 10, 20 e 40 SM. Prima delle misure si è verificato, dai tempi di esecuzione, che le percentuali del 5% e del 25% concedono effettivamente 1 e 5 TPC. Il tempo misurato è il tempo di calcolo dell'intero programma nella versione F5 (10 iterazioni, mediana di 2 esecuzioni), con $K = 64$, $NC = 512$ e $DEG = 3$. Si osservi che MPS limita gli SM, ma non le risorse condivise tra essi, cioè la cache L2 e la banda verso la memoria globale.

Dimensione del problema. Per lo scaling forte si usa $N = 12800$ fisso. Per lo scaling debole il lavoro per processore deve restare costante: poiché il calcolo delle forze, che costituisce la gran parte del tempo, ha costo $O(N^2 \cdot K)$ per iterazione, il rapporto N^2/P deve restare costante, cioè N deve crescere come $\sqrt{P}$:

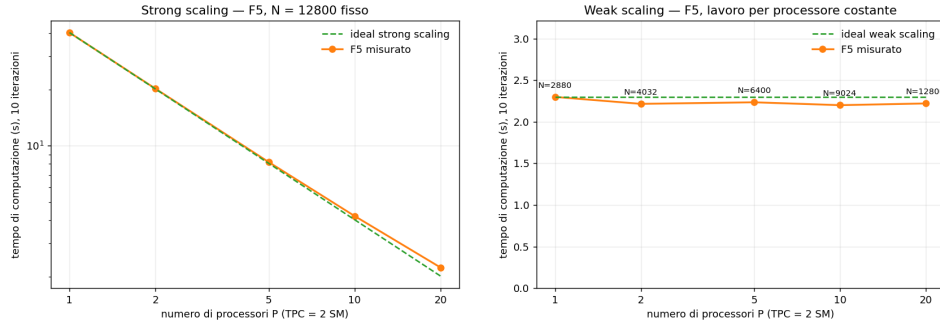
$$N(P) = 12800 \cdot \sqrt{P/20} \quad (21)$$

arrotondato al multiplo di 64 più vicino, da cui $N = 2880, 4032, 6400, 9024, 12800$. Lo speedup scalato si calcola sul lavoro effettivamente svolto, $S(P) = (N_P^2/N_1^2) \cdot T_1/T_P$, che corregge anche l'effetto dell'arrotondamento. La Tabella 3 e la Figura 2 riportano i risultati.

Scaling forte. Nella Figura 2a la curva misurata segue quasi esattamente la

P	SM	Forte ($N = 12800$)				Debole			
		T_P (s)	S	E	s	N	T_P (s)	S	E
1	2	40,282	1,00	1,00	—	2880	2,298	1,00	1,00
2	4	20,230	1,99	1,00	0,44%	4032	2,215	2,03	1,02
5	10	8,181	4,92	0,98	0,39%	6400	2,234	5,08	1,02
10	20	4,215	9,56	0,96	0,52%	9024	2,199	10,26	1,03
20	40	2,238	18,00	0,90	0,59%	12800	2,220	20,45	1,02

Tabella 3: Scalabilità forte e debole della versione F5 (tempo di calcolo dell'intero programma, 10 iterazioni). P è il numero di processori (TPC), S lo speedup (scalato, nel caso debole), E l'efficienza e s la porzione seriale ricavata dalla legge di Amdahl.



(a) Scaling forte, $N = 12800$ fisso (scala logaritmica). (b) Scaling debole, lavoro per processore costante.

Figura 2: Tempo di computazione della versione F5 in funzione del numero di processori (TPC), confrontato con l'andamento ideale (tratteggio).

retta ideale T_1/P : lo speedup arriva a 18 con 20 processori, con efficienza 0,90. Lo scostamento dall'ideale è spiegato dalla legge di Amdahl. La porzione seriale stimata resta tra lo 0,4% e lo 0,6% per tutti i valori di P , quindi si comporta come una vera porzione seriale del programma. Con $s = 0,005$ la legge prevede $S = 4,90, 9,57$ e $18,26$ per $P = 5, 10$ e 20 , contro i valori misurati $4,92, 9,56$ e $18,00$. In tempo assoluto questa porzione vale circa 0,2 s su 40 ed è compatibile con le operazioni che non dipendono dal numero di SM: quelle svolte sull'host (generazione dei numeri casuali, riduzioni, selezione di K_{best}), i trasferimenti fra host e device e il lancio dei kernel (Sezione 2.3). Per la stessa legge, lo speedup non potrebbe superare $1/s \approx 200$ nemmeno con un numero illimitato di processori.

Scaling debole. Nella Figura 2b il tempo resta compreso tra 2,20 e 2,30 s mentre il lavoro complessivo cresce di 20 volte: la curva coincide con la retta ideale e l'efficienza è pari a 1 entro il 3%. I valori leggermente superiori a 1

rientrano nella variabilità delle misure, e la porzione seriale ricavata dalla legge di Gustafson, $s = (P - S)/(P - 1)$, è nulla entro la stessa incertezza. **Confronto tra le due leggi.** I due esperimenti mostrano la differenza tra le leggi di Amdahl e di Gustafson. La stessa piccola porzione seriale riduce l'efficienza forte a 0,90 con 20 processori, ma non intacca quella debole. A dimensione fissa, infatti, la parte parallela di ogni processore si riduce al crescere di P , mentre quella seriale resta invariata e il suo peso relativo aumenta. Quando invece il lavoro cresce insieme ai processori, la parte parallela di ogni processore resta costante e il peso della porzione seriale non cambia.

3.5 Confronto tra due GPU: Tesla T4 e GeForce RTX 4060

Per rendere il progetto riproducibile con un unico comando è stato realizzato un `Makefile` (Sezione 4.1), sviluppato e provato sui PC dei laboratori dell'Università (aula A028). Su queste macchine è stato ripetuto il confronto tra le cinque versioni del kernel delle forze della Sezione 3.3, che ha mostrato risultati diversi da quelli ottenuti sulla Tesla T4. Questa sezione riporta le nuove misure e spiega da dove nascono le differenze.

Ambiente. I PC del laboratorio montano una NVIDIA GeForce RTX 4060, architettura Ada Lovelace, compute capability 8.9. Il codice è eseguito in Ubuntu tramite il Windows Subsystem for Linux (WSL), con CUDA 13.1; i sorgenti sono gli stessi, compilati con `nvcc -O2 -arch=native`. La Tabella 4 confronta le caratteristiche delle due GPU che risultano rilevanti per questo studio.

	Tesla T4	GeForce RTX 4060
Architettura (cc)	Turing (7.5)	Ada Lovelace (8.9)
SM	40	24
Frequenza massima	circa 1,6 GHz	circa 2,5 GHz
Cache L2	4 MB	24 MB
Banda verso la memoria	320 GB/s	272 GB/s
Doppia precisione	circa 0,25 TFLOPS	circa 0,24 TFLOPS

Tabella 4: Caratteristiche delle due GPU utilizzate (valori nominali dei produttori; la doppia precisione è 1/32 della singola sulla T4 e 1/64 sulla RTX 4060).

Confronto tra le cinque versioni. Lo sweep su N è stato ripetuto con lo stesso script e la stessa taglia ($K = 64$, $NC = 512$, $DEG = 3$), estendendolo fino a $N = 25600$. La Tabella 5 riporta i tempi del kernel delle forze sulla RTX 4060 e la Figura 3 il relativo grafico. Per agevolare il confronto, la Tabella 6 riporta per ogni versione il rapporto tra il tempo sulla T4 e quello sulla RTX 4060: un valore maggiore di 1 indica che la versione è più veloce

sulla RTX 4060.

N	F1	F2	F3	F4	F5	F1/F5
100	0,402	0,308	0,057	0,059	0,035	11,5×
200	1,354	1,068	0,174	0,187	0,130	10,4×
400	3,042	2,364	0,394	0,428	0,330	9,2×
800	5,986	4,633	1,098	1,207	0,986	6,1×
1600	11,809	9,064	3,603	4,030	3,409	3,5×
3200	23,728	17,880	12,944	14,407	12,456	1,9×
6400	94,423	72,220	51,102	57,041	50,222	1,9×
12800	284,459	216,210	200,413	224,077	198,233	1,4×
25600	936,437	712,644	785,737	879,292	785	1,2×

Tabella 5: Tempo del solo kernel delle forze sulla GeForce RTX 4060 (ms per iterazione, mediana di 2 esecuzioni da 100 iterazioni). In grassetto la versione più veloce a ogni taglia.

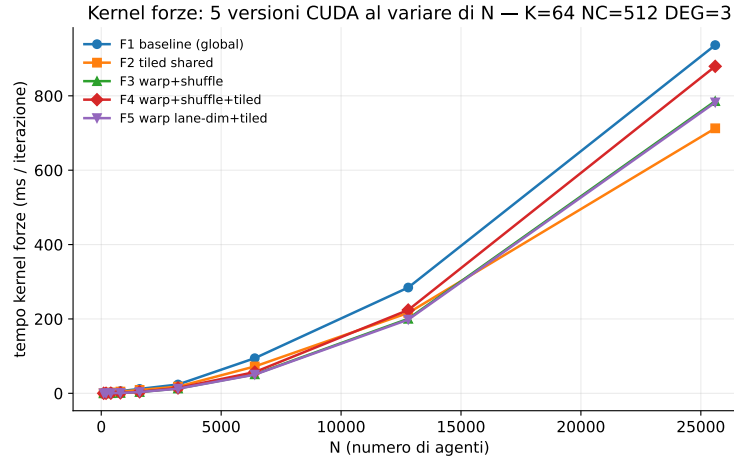


Figura 3: Tempo del kernel delle forze al variare di N per le cinque versioni CUDA sulla GeForce RTX 4060.

Fino a $N = 12800$ il quadro generale coincide con quello della T4: le versioni a warp sono un ordine di grandezza più veloci della baseline alle taglie piccole, il vantaggio si riduce al crescere di N e F5 è la versione più veloce. Le differenze sono però marcate, e la Tabella 6 permette di attribuirle a tre caratteristiche dell'hardware.

Doppia precisione: F5 va alla stessa velocità. La versione F5 impiega praticamente lo stesso tempo sulle due GPU (rapporto tra 0,98 e 1,03 da $N = 1600$ in poi). F5 lavora quasi interamente in shared memory, con accessi coalescenti e privi di conflitti di banco: il suo tempo è dominato

N	F1	F2	F3	F4	F5
800	1,82	1,69	1,71	1,14	1,20
1600	1,78	1,74	1,67	1,11	0,98
3200	1,72	1,70	1,74	1,11	1,02
6400	1,54	0,93	1,82	1,11	1,02
12800	2,65	1,38	1,81	1,12	1,03
25600	2,61	1,31	1,88	1,12	1,02

Tabella 6: Rapporto tra il tempo del kernel delle forze sulla Tesla T4 e sulla GeForce RTX 4060 (valori maggiori di 1: più veloce sulla RTX 4060). Per $N \leq 6400$ i tempi della T4 sono quelli della Tabella 2; per $N = 12800$ e $N = 25600$ provengono da un'esecuzione dello stesso script in una sessione Colab distinta.

dalle operazioni in doppia precisione, e le due GPU ne hanno una capacità complessiva quasi uguale (circa 0,25 TFLOPS). Lo stesso vale, in misura minore, per F4 (rapporto 1,11).

Frequenza degli SM: F1, F2 e F3 sono più veloci a N piccolo. Fino a $N = 3200$ le versioni F1, F2 e F3 sono da 1,7 a 1,8 volte più veloci sulla RTX 4060. F1 e F2 usano un thread per agente e quindi pochi blocchi: il loro tempo dipende dalla velocità del singolo SM, che sulla RTX 4060 lavora a una frequenza più alta (circa 2,5 GHz contro 1,6). F3 legge invece i sorgenti dalla memoria globale con accessi non coalescenti, e beneficia di una gerarchia di memoria più rapida.

Cache L2: F1 e F3 sono molto più veloci a N grande. Il kernel delle forze rilegge continuamente l'array delle posizioni e scrive quello delle accelerazioni, che insieme occupano $2 \cdot N \cdot K \cdot 8$ byte: 6,6 MB a $N = 6400$, 13 MB a $N = 12800$, 26 MB a $N = 25600$. Sulla T4, con 4 MB di L2, i due array escono dalla cache già a $N = 6400$ e le versioni che leggono dalla memoria globale rallentano più del previsto: è il salto della baseline tra $N = 3200$ e $N = 6400$ discusso nella Sezione 3.3, e tra 6400 e 12800 la baseline cresce ancora di 5,2 volte invece di 4. Sulla RTX 4060, con 24 MB di L2, i due array restano in cache fino a $N = 12800$ e la baseline cresce in modo regolare. Per questo il rapporto tra le due GPU per F1 sale fino a 2,6 alle taglie grandi, nonostante la RTX 4060 abbia una banda verso la memoria globale inferiore. La cache più grande spiega anche due comportamenti presenti solo sulla RTX 4060:

- *F4 è più lenta di F3 a tutte le taglie.* Il tiling in shared memory serve a evitare letture ripetute dalla memoria globale; se queste sono già servite dalla L2, il risparmio si riduce e resta il costo delle barriere `__syncthreads()` e dei conflitti di banco nella lettura del tile (Sezione 2.4).

- A $N = 25600$ la versione più veloce è F2. Con 25600 agenti anche le versioni a un thread per agente lanciano abbastanza blocchi da occupare tutti gli SM, quindi il vantaggio della decomposizione a warp scompare. Resta il modo di accedere ai dati: in F2 tutti i thread leggono lo stesso elemento della shared memory, in broadcast, e il caricamento dei tile è accelerato dalla L2, mentre F5 è limitata dalla doppia precisione e non guadagna nulla dalla cache. Sulla T4 F5 resta invece la più veloce anche a questa taglia (800 ms contro 935 di F2).

Un ultimo dettaglio riguarda il rapporto tra blocchi e SM. Con 256 thread per blocco, a $N = 6400$ F1 e F2 lanciano 25 blocchi: sulla T4 trovano posto tutti contemporaneamente sui 40 SM, sulla RTX 4060 i 24 SM non bastano e uno di essi deve eseguire due blocchi in sequenza, raddoppiando il tempo dell'intero kernel. È il motivo per cui, solo a questa taglia, F2 è più lenta sulla RTX 4060 (rapporto 0,93) e la baseline passa da $N = 3200$ a $N = 6400$ con un fattore 4 invece di 2.

In sintesi, il confronto mostra che **il vantaggio relativo delle ottimizzazioni dipende dall'architettura**. Tiling e accessi ottimizzati alla shared memory rendono di più quando la memoria globale è il collo di bottiglia, come sulla T4 con la sua L2 da 4 MB; su una GPU con una L2 molto più grande quel vantaggio si riduce. La decomposizione a warp, che distribuisce il lavoro su tutti gli SM anche con pochi agenti, resta invece la leva più efficace su entrambe le GPU alle taglie piccole, e F5 rimane la versione più veloce fino a $N = 12800$ su entrambe.

4 Valutazioni e osservazioni

L'implementazione presenta diversi punti di forza. Il primo è lo studio approfondito del kernel dominante: le cinque versioni del calcolo delle forze mostrano, con misure dirette, l'effetto delle principali tecniche di ottimizzazione (tiling in shared memory, decomposizione a livello di warp, riduzione con warp shuffle, accessi coalescenti e privi di conflitti di banco) e ne quantificano il contributo, portando a un miglioramento del kernel da circa 3 a oltre 20 volte a seconda della taglia. Il secondo è la strategia di confronto: misurando il tempo del solo kernel tramite gli eventi CUDA e tenendo fisso tutto il resto della dinamica, l'effetto di ciascuna ottimizzazione è isolato in modo pulito. Il terzo è la correttezza verificabile: poiché la valutazione di funzione obiettivo e vincoli avviene in aritmetica intera, le fitness della versione seriale e di quella CUDA coincidono esattamente, e tutte le versioni del kernel delle forze producono la stessa soluzione finale (le versioni a decomposizione warp possono differire solo per l'ultima cifra dei valori intermedi, per la non associatività della virgola mobile). Il quarto è la scalabilità della versione finale: F5 raggiunge un'efficienza di 0,90 nello scaling forte su 20 processori, scostamento spiegato da una porzione seriale di circa lo 0,5%, e un'efficienza

pari a 1 nello scaling debole. Il quinto è la verifica su due architetture diverse: il confronto tra le cinque versioni ripetuto su una GeForce RTX 4060 conferma la correttezza dei risultati e mostra come il vantaggio di ciascuna ottimizzazione dipenda dalla gerarchia di memoria e dalla capacità in doppia precisione della GPU (Sezione 3.5).

Non mancano i limiti. A ogni iterazione si eseguono trasferimenti fra host e device di grandezze di costo $O(N)$ e le riduzioni per *best*, *worst* e la somma delle masse restano sull'host: a N piccolo questo overhead pesa relativamente di più sul tempo complessivo. Queste operazioni non dipendono dal numero di SM e costituiscono la porzione seriale che limita lo scaling forte (Sezione 3.4). Inoltre l'accumulatore locale per agente, un array di K elementi, grava sulla pressione dei registri e nelle versioni F1–F4 limita a un solo blocco quelli che possono risiedere su ogni SM; solo F5, che lo riduce a due registri per lane, ne ospita due. Tutto il calcolo delle forze avviene in doppia precisione, che sulla Tesla T4 ha un throughput 32 volte inferiore alla singola; inoltre la versione finale calcola ogni differenza tra coordinate due volte, una per fase. Tra le possibili miglorie si segnalano diverse direzioni. La prima è ridurre le operazioni in doppia precisione di F5 riscrivendo l'accumulo come $\sum_j c_j x_j^d - x_i^d \sum_j c_j$, che elimina la seconda sottrazione per dimensione; il risultato non sarebbe più identico bit a bit alla baseline e andrebbe valutato il rischio di cancellazione numerica quando i coefficienti sono grandi. La seconda è svolgere il calcolo delle forze in singola precisione, che sulla T4 ha un throughput 32 volte superiore: le traiettorie divergerebbero da quelle della versione seriale, ma poiché la fitness è valutata sulle posizioni arrotondate a interi si potrebbe verificare sperimentalmente se la qualità delle soluzioni cambia. La terza è spostare sul device anche le riduzioni per *best*, *worst* e somma delle masse, così da eliminare i relativi trasferimenti a ogni iterazione, ridurre l'overhead alle taglie piccole e ridurre la porzione seriale che limita lo scaling forte. L'ultima, quando NC è sufficientemente piccolo da rientrare nel limite di 64 KB, è collocare le tabelle dei coefficienti dei vincoli in constant memory, adatta a dati di sola lettura letti in modo uniforme da tutti i thread; con la taglia usata in questo studio, però, le tabelle superano tale limite e restano quindi in global memory.

4.1 Mini-guida all'uso

Il progetto si compila ed esegue tramite il `Makefile`, da lanciare in un terminale Linux (per esempio Ubuntu sotto WSL) nella cartella che contiene i sorgenti. I target principali sono:

```
make          # compila seriale e cinque versioni CUDA
make demo     # prova rapida: seriale, F1, F4, F5
make clear    # cancella eseguibili e temporanei
```

La taglia dell'istanza è fissata a tempo di compilazione; i valori predefiniti sono quelli usati in questa relazione ($K = 64$, $NC = 512$, $DEG = 3$) e si

possono cambiare da riga di comando. Il codice CUDA è compilato per la GPU presente con `-arch=native`, oppure per un'architettura specifica:

```
make clear && make N=3200 MAX_ITER=200 # altra taglia
make ARCH=sm_75                        # Tesla T4
```

Il seme delle funzioni è fisso nel sorgente, quindi a parità di taglia l'istanza e l'esecuzione sono riproducibili.

Gli esperimenti della Sezione 3 si riproducono con i target seguenti, che richiedono Python 3 con matplotlib:

```
make sweep          # cinque versioni al variare di N
make scaling        # scalabilità forte e debole di F5
make scaling-csv    # figure di scalabilità dal CSV
```

Il target `sweep` esegue lo script `benchmark_cuda.py`, che compila le cinque versioni con `-DPROFILE`, misura con gli eventi CUDA il solo kernel delle forze (mediana di 2 esecuzioni) e genera la figura in formato PDF e PNG. Il target `scaling` esegue lo script `scaling_cuda.py`, che legge il numero di SM della GPU, e genera le due figure di scalabilità. Poiché MPS non è disponibile sotto WSL, lo script limita i processori in un altro modo: con la direttiva `-DNBLK=P` i kernel di fitness, forze (F5) e aggiornamento si lanciano con esattamente P blocchi, ciascuno assegnato a un SM diverso, e scorrono tutti gli agenti con un ciclo *grid-stride*; con la griglia normale il ciclo esegue una sola passata e le altre misure non cambiano. In questo caso il processore è il singolo SM. Le misure della Sezione 3.4 sono state invece ottenute sulla Tesla T4 con MPS, disponibile nell'ambiente Colab.

I singoli programmi si possono anche compilare a mano. La versione seriale:

```
gcc -O2 -DN=1600 -DK=64 -DNC=512 -DDEG=3 -DMAX_ITER=100 \
-o gsa_serial gsa_serial.c -lm
```

La versione CUDA si compila selezionando la versione del kernel delle forze con una direttiva `-D`; senza flag si ottiene la baseline (F1):

```
nvcc -O2 -arch=native          gsa_cuda.cu -o gsa_cuda # F1
nvcc -O2 -arch=native -DTILED  gsa_cuda.cu -o gsa_cuda # F2
nvcc -O2 -arch=native -DFORCE_WARP gsa_cuda.cu -o gsa_cuda # F3
nvcc -O2 -arch=native -DFORCE_WARP_TILED gsa_cuda.cu -o gsa_cuda # F4
nvcc -O2 -arch=native -DFORCE_WARP_DIM gsa_cuda.cu -o gsa_cuda # F5
```

Aggiungendo `-DPROFILE` il programma stampa, tramite gli eventi CUDA, il tempo per iterazione del kernel delle forze e di quello della fitness separatamente. L'uso dei registri e della local memory di ogni kernel si legge compilando con `-Xptxas -v`.

Riferimenti bibliografici

- [1] E. Rashedi, H. Nezamabadi-pour, S. Saryazdi, *GSA: A Gravitational Search Algorithm*, Information Sciences, vol. 179, n. 13, pp. 2232–2248, 2009.